

LAB REPORT / ROBOT VISION

Linux, Git, C++ and CMake Fundamentals

Command-line practice and a RandomVector implementation

Name	Zhang Linjie	Student ID	24020036086
Date	September 29, 2026	Course	Robot Vision

01 Experiment Overview

This experiment introduces the Linux command line, Git, C++ and CMake. The work was completed on Ubuntu 22.04. A public course repository was cloned, text-processing and output-redirection exercises were completed, and the supplied `RandomVector` class was implemented without using algorithms from the `<algorithm>` header.

All required programs compiled and ran successfully. The C++ program generated 20 reproducible random values, calculated their mean and extrema, and printed a five-bin histogram. A separate CMake demonstration verified a multi-file executable linked against a custom library.

Task	Result	Conclusion
Shell exercises	Completed	Output files verified
RandomVector	Compiled and executed	Statistics correct
CMake demo	Compiled and executed	Library linked successfully

02 Experimental Environment

Item	Configuration
Operating System	Ubuntu 22.04 LTS
C++ Compiler	g++ 11.4.0
CMake	3.22.1
Git	2.34.1
Build Standard	C++11 with <code>-Wall -pedantic</code>

Ubuntu 22.04 is sufficient for the Shell, Git, C++ and CMake content in Lab 1. If later laboratories require ROS 1 Noetic, a separate Ubuntu 20.04 environment should be used to match the course dependencies.

03 Git and Shell Exercises

3.1 Course Repository

The course repository was cloned from <https://github.com/ouc-vlab-course/vnav-codes>. The first attempt failed because of an HTTP/2 framing error. Retrying with HTTP/1.1 and a shallow clone completed successfully.

```
git -c http.version=HTTP/1.1 clone --depth 1 \
https://github.com/ouc-vlab-course/vnav-codes.git
```

3.2 Text Statistics

The file `dante.txt` was analysed with `wc` and `grep`. The first result line used `>` to create the output file, while subsequent lines used `>>` to append results.

Metric	Result
Total lines	19,567
Total words	97,676
Nonblank lines	14,338

```
echo "Lines: $(wc -l < dante.txt)" > exercise1.txt
echo "Words: $(wc -w < dante.txt)" >> exercise1.txt
echo "Nonblank lines: $(grep -cv '^[[:space:]]*$' dante.txt)" \
>> exercise1.txt
```

3.3 Append Redirection

The `fortune` command was run five times. Every result was appended to `fortunes.txt`; the final file contained five lines. This demonstrates that `>>` preserves existing data, unlike `>`, which overwrites it.

04 C++ Fundamentals

Question	Result
Prefix and postfix increment	$i = 2, j = 1$
Conditional operator	Prints b
Reference assignment	Prints 10 10
Pointer multiplication	Prints 1764
Pointer and array indexing	Prints 0
Reference parameter reset	Prints 0
Conversion of 3.14 to int	Stores 3
Conversion of -1 to 8-bit unsigned char	Stores 255
Integer used as a condition	Final value is 0

05 RandomVector Implementation

5.1 Methods

The constructor fills a vector with random values between zero and `max_val`. The `mean()`, `min()` and `max()` methods use explicit loops. The histogram divides the interval from the minimum to the maximum into equal-width bins. A value equal to the maximum is assigned to the final bin to prevent an out-of-range index.

The implementation also checks invalid vector sizes, negative maximum values and non-positive bin counts. No function from `<algorithm>` is used.

```
g++ -std=c++11 -Wall -pedantic \
    -o random_vector main.cpp random_vector.cpp
./random_vector
```

5.2 Experimental Results

The program uses the fixed seed 314159, making the sequence reproducible on the same C library implementation.

Metric	Result
Number of samples	20
Mean	0.558040
Minimum	0.0278694
Maximum	0.982655
Five-bin counts	5, 1, 4, 3, 7
Remaining TODO markers	0

Histogram:

```

          ***
          ***

***  ***
***  ***  ***
***  ***  ***  ***
***  ***  ***  ***
***  ***  ***  ***  ***
```

06 CMake Multi-file Build

The demonstration project contains `main.cpp`, `my_lib.cpp`, `my_lib.hpp` and `CMakeLists.txt`. CMake creates the `mylib` library, builds the `main` executable and links the executable to the library.

```
mkdir build && cd build
cmake ..
cmake --build .
./main

Hello world!
In library
```

The output confirms that the executable called the function implemented in the separate library source file.

07 Analysis and Discussion

The main setup difficulty was the interrupted GitHub connection. Forcing HTTP/1.1 avoided the HTTP/2 framing error without changing the repository. This demonstrates that a failed clone does not necessarily indicate an incorrect Git installation.

The most important programming boundary case occurred in histogram binning. The direct expression for a maximum-valued sample can produce an index equal to the number of bins. Since valid indices end at `bins - 1`, the implementation explicitly maps this value to the final bin. Checks for invalid constructor arguments and histogram parameters further improve robustness.

The measured Shell results are deterministic for the downloaded text. The RandomVector results are reproducible within the current environment because the program uses a fixed seed. Exact pseudo-random sequences can differ between C library implementations, so the algorithm and statistical properties are more important than matching every generated value on another platform.

08 GitHub Repository and Commit History

The completed exercises and source code are maintained in the personal public repository <https://github.com/Eva122205/vnav-personal>. The Lab 1 materials are located in the `lab1/` directory. Work was recorded through multiple Git commits instead of a single final upload. Figure 1 shows the repository commit history at the time of submission.

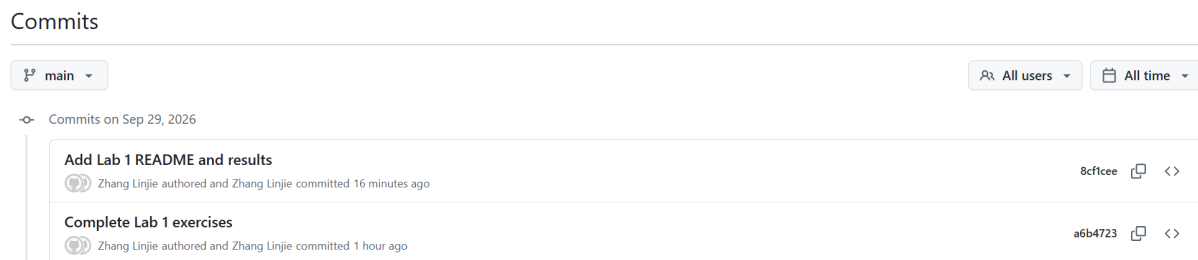


图 1: GitHub commit history for Lab 1, including commit identifiers 8cf1cee and a6b4723.

09 Conclusion

This experiment successfully completed the introductory Linux, Git, C++ and CMake tasks. Shell commands produced the required text statistics and append-only output file. The C++ questions reinforced expression evaluation, references, pointers and numeric conversion. RandomVector compiled without warnings, calculated the expected statistics and produced a five-bin histogram. Finally, the CMake demonstration generated and built a multi-file project linked to a custom library.

The results show that Ubuntu 22.04 can complete every Lab 1 task. The complete work is stored under `/vnav-personal/lab1` for later submission.

10 Submission Record

Item	Record
Personal Repository	https://github.com/Eva122205/vnav-personal
Course Repository	https://github.com/ouc-vlab-course/vnav-codes
Commit Records	8cf1cee, a6b4723
Shell Results	exercise1.txt, fortunes.txt
Written Answers	cpp_answers.txt
C++ Program	RandomVector/
CMake Program	lab1_demo/